| INSTITUCIÓN UNIVESITARIA DE COLOMBIA | |
|---|---|
| PROGRAMA ACADÉMICO DE INGENIERÍA DE SOFTWARE | |
| CONTENIDO PROGRAMÁTICO | |
| DATOS GENERALES ASIGNATURA | |
| **Asignatura:** | PRUEBAS DE SOFTWARE |
| **Periodo Académico:** | VII Semestre |
| **Créditos:** | 3 |
| **Duración en Horas:** | 40 |

## I. JUSTIFICACIÓN

Muchas veces, cuando trabajamos de manera independiente en proyectos pequeños, no tenemos la necesidad de (o el cliente no cuenta con el presupuesto para) escribir pruebas automatizadas.

Sin embargo, al empezar a trabajar en proyectos más grandes e integrarnos a equipos de trabajo más amplios, notamos que muchas veces se habla de distintos tipos de tests.

## II. OBJETIVOS

**Objetivo General:**

**Las pruebas manuales** son llevadas a cabo por personas, quienes navegan e interactúan con el software (usando herramientas adecuadas para cada caso).

Instruir a los estudiantes el mundo metodológico de la materia Calidad y Métricas de Software relacionándolos con diferentes tipos de metodologías, tipos depruebas y diferentes Métricas de calidad del Software

**Objetivos Específicos**:

- ✓ Conocer la importancia de establecer alcances claros y reales de un proyecto de Software


© Todos los derechos reservados (2024).

- ✓ Conocer y entender la importancia de las metodologías en el proceso de construcción de Software.

- ✓ Aprender a levantar y especificar requerimientos para proyectos deSoftware.
- ✓ Entender la importancia construir modelos y establecer comunicación entrelos diferentes actores de un proyecto por medio de estos modelos.
- ✓ Aprender sobre Arquitecturas para la construcción de Sistemas, bajomodelos cliente servidor o ambientes empresariales.
- ✓ Aprender a diseñar modelos y especificar su estructura y comportamiento.
- ✓ Conocer nuevas tecnologías y nuevas herramientas para construcción y/o pruebas de software.
- ✓ Aprender a interpretar los modelos de Métricas existentes.

- ✓ Aprender o reafirmar los conocimientosen el campo de la programación orientada a objetos.
- ✓ Realizar el Diseño y la implementación de un prototipo funcional como trabajo final, que permita aplicar los conocimientos adquiridos en los diferentes campos, tales como metodología, modelado con UML, desarrollo con lenguajes de programación orientados a objetos, así comocon el manejo de herramientas para desarrollo y/o modelado y generar todos los tipos de pruebas existentes según la métrica utilizada


© Todos los derechos reservados (2024).

| III. TEMARIO POR SESIONES | RESULTADOS DE APRENDIZAJE POR SESIÓN |
|---|---|
| **Sesión 1.: Introducción (2 horas)**<br>• **Contenidos Programático**<br>• Manual vs Automated testing<br>a. **Las pruebas manuales**<br>b. **Las pruebas automatizadas** | • Socializar el contenido programático y metodología de evaluación. |
| **SESION 2 – 5**<br>• **Los diferentes tipos de tests**<br>• Unit tests<br>o Integration tests<br>o Test Driven Development<br>• Integration tests<br>o Las **pruebas de integración**<br>• Functional tests<br>o Las **pruebas de punta a punta** | • Identificar los productos del Software la evolución sus características. |
| **SESION 6 - 7**<br>• Regression testing<br>o Las **pruebas de regresión**<br>• Smoke testing<br>o Las **pruebas de humo**<br>• Acceptance testing<br>o Las **pruebas de aceptación** | • Dar a conocer al estudiante los conceptos de la métrica. Y su clasificación para que identifique fácilmente el procedimiento de verificación |
| **SESION 8 – 9**<br>• **Performance testing**<br>o **Las pruebas de rendimiento**<br>• **Por qué y cómo automatizar nuestras pruebas**<br>o **servidor de continuous integration.**<br>o **servicio en la nube** | • Reconocer los Métodos CK Chidamber Metricas Kemerer. |
| **SESION 10**<br>• **Primer parcial Evaluacion de conocimientos** | • Evaluar los conocimientos adquiridos. |
| **SESION 11 - 14**<br>• **Exploratory testing** | • Identificar los modelos de procesamiento mediante ejercicios |


© Todos los derechos reservados (2024).

| | |
|---|---|
| • **Programación de Testing y Dinamica**<br>• **Generalidades de la programación dinámica**<br>  • **Resolución de un problema de programación.**<br>  • **Terminología**<br>**SESION 15**<br>  • **Segundo Parcial Evaluación de Conocimientos**<br>**SESION 16 - 19**<br>  • **Etapas y estados en programación dinámica**<br>  ○ **Algoritmos de solución**<br>  ○ **El problema de la mochila**<br>  ○ **Cálculo de los números de Fibonacci**<br>  ○ **Calculo de los coeficientes binomiales**<br>  ○ **La subsecuencia común máxima**<br>  ○ **El problema del camino de mínimo costo**<br>  ○ **Asignación de recursos**<br>**SESION 20**<br>1. **Examen final (Evaluación de Conocimientos)** | y descripción por parte del estudiante.<br><br>• Aplicar Técnicas para el análisis, la calidad del software modelado y validación del mismo<br><br>• Evaluar los conocimientos adquiridos.<br><br>• Trabajar con aprendizaje basado en comienzos de la programación<br><br>• Aplicar los conceptos teóricos a una situación real, con el fin de dar solución a una necesidad a través de herramientas de programación y como dar solución |


© Todos los derechos reservados (2024).

## IV. COMPETENCIAS

- Plantear la resolución de problemas de ingeniería mediante el uso de las matemáticas.
- Comprobar mediante la abstracción y el análisis, fenómenos físicos
- Construir algoritmos y programas de computación para analizar y evaluar problemas básicos de medición.
- Aplicar métodos staandares de la métrica para solucionar problemas de software.
- Aplicar dentro del ejercicio profesional el uso de un segundo idioma.
- Aplicar los principios de la ética en el comportamiento ciudadano y en el ejercicio profesional de la Ingeniería de Software.
- Aplicar habilidades en el uso de las tecnologías de la información y la comunicación
  - Desarrollar trabajo en equipo aplicando los conocimientos de manera práctica y con alta calidad.
- Utilizar la estadística y la probabilidad para analizar e interpretar los resultados de procesos experimentales y observacionales relacionados con la ingeniería. profesión.
- Tomar decisiones que contribuyan al correcto desempeño de su ejercicio laboral.

## V. RESULTADOS DE APRENDIZAJE

- Identificar el contexto sobre el cual se apoya la Ingeniería de Software en relación a métodos, metodologías y herramientas que permitan el manejo adecuado de los procesos de la métrica.
- Aplicar procesos de fundamentos de que indica la métrica, utilizando herramientas adecuadas para la extracción y tratamiento de los requerimientos de evaluación y verificación del software.

- Especificar los requisitos básicos para una evaluación de software para su validación, documentación y herramientas.
- Identificar los diferentes niveles
- Entender la importancia de estudiar la ingeniería de Software, identificando como esta en mercado actual y su perfil laboral


© Todos los derechos reservados (2024).

## VI. ESTRATEGIAS DE APRENDIZAJE

Se implementa un diseño que ayude a la obtención de resultados de calidad, donde se dirija al estudiante a la motivación de sus sentidos, estimulando su participación en la construcción de saberes y de valores, que fomenten su capacidad crítica y de análisis, obteniendo bajo esta perspectiva pedagógica un individuo propositivo y coherente con las nuevas exigencias del mundo, formado bajo presupuestos de la la CIBERSEGURIDAD, sabiendo utilizar sus recursos y con ello siendo eficiente y eficaz.

Prueba diagnóstica (conducta de entrada) aplicada al grupo al iniciar el curso.
Ejemplos prácticos y probados desde referentes internacionales y desde la experiencia e investigación del docente.

Entrega de materiales de acuerdo con el desarrollo de la estructura de contenidos, que conllevan una intencionalidad pedagógica centrada en el aprendizaje tanto en el TP como en el TI.
Introducciones reflexivas y análisis de posturas críticas mediante lecturas de artículos de expertos o hechos noticiosos que ayudan a dar elementos de aprehensión del conocimiento y visión de conjunto.

## VII. METODOLOGÍA

**Trabajo Presencial:**

- Clase magistral dada por el docente.
- Exposiciones con apoyo del docente.
- Parciales o Exámenes Orales.
- Seminarios o Conferencias orientadas por el docente.

**Trabajo Dirigido:**

- Prácticas de laboratorio.
- Talleres y ensayos.
- Tutorías.

**Trabajo Independiente:**

- Elaboración de los informes de laboratorio.
- Lecturas e investigación de aula.
- Elaboración de talleres y exposiciones
- Presentación de exámenes escritos o virtuales.


© Todos los derechos reservados (2024).

## VIII. EVALUACIÓN

Se definen tres (3) momentos de evaluación citados en el desarrollo de las sesiones,así:

SESIÓN 1 - 8
- ✓ Primera evaluación 30 %. Teórico práctica.

SESIÓN 9 - 14
- ✓ Segunda evaluación 30 %. Teórico práctica.

SESIÓN 20
- ✓ Tercera evaluación 40 %. Teórico práctica.

## IX. BIBLIOGRAFÍA


- Different types of testing explained
- The different types of software testing
- Rumbaugh J., Jacobson I., Booch G. El Lenguaje Unificado de Modelado.Manual de Referencia. Editorial Addison-Wesley – 2000
- Braude Eric. Ingeniería de Software. Una perspectiva orientada a objetos.Alfaomega Grupo Editor S.A. 2003.
- Daniel Bolaños, Almudena Sierra, Idoia Alarcón. Pruebas de software y JUnit.Un análisis en profundidad y ejemplos prácticos. Pearson Education. 2007.
- Booch G; Rumbaugh J; Jacobson I. El Lenguaje Unificado de Modelado.



© Todos los derechos reservados (2024).

UML
2.0 Addison Wesley. 2006.

- Larman Craig. UML y Patrones. Prentice Hall. 1999.
  [MEY99] Meyer Beltrand. Construcción de software Orientado a Objetos. Segunda Edición. Prentice Hall. 1999.


© Todos los derechos reservados (2024).